192425037

EXP20:

```
#include <stdio.h>

#include <string.h>

#include <stdint.h>


static const int
IP[64]={58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7};

static const int
FP[64]={40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25};

static const int
EX[48]={32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,12,13,12,13,14,15,16,17,16,17,18,19,20,21,20,21,22,23,24,25,24,25,26,27,28,29,28,29,30,31,32,1};

static const int
PP[32]={16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25};

static const int
PC1[56]={57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,59,51,43,35,27,19,11,3,60,52,44,36,63,55,47,39,31,23,15,7,62,54,46,38,30,22,14,6,61,53,45,37,29,21,13,5,28,20,12,4};

static const int
PC2[48]={14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,26,8,16,7,27,20,13,2,41,52,31,37,47,55,30,40,51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32};

static const int SHIFTS[16]={1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1};

static const int SB[8][64]={

{14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7,0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8,4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0,15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13},

{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10,3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5,0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15,13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9},

{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8,13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1,13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7,1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12},

{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15,13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9,10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4,3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14},
```

```
{2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9,14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6,4,2,1,11,10,13,7,
8,15,9,12,5,6,3,0,14,11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3},

{12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11,10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8,9,14,15,5,2,8,12,
3,7,0,4,10,1,13,11,6,4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13},

{4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1,13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6,1,4,11,13,12,3,7,
14,10,15,6,8,0,5,9,2,6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12},

{13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7,1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2,7,11,4,1,9,12,14,
2,0,6,10,13,15,3,5,8,2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}};


static void permute(const int *t,int n,const unsigned char *in,unsigned char *out){
   int i; for(i=0;i<n;i++) out[i]=in[t[i]-1];
}
static void to_bits(const unsigned char *b,int n,unsigned char *bits){
   int i; for(i=0;i<n*8;i++) bits[i]=(b[i/8]>>(7-i%8))&1;
}
static void from_bits(const unsigned char *bits,int n,unsigned char *b){
   int i; memset(b,0,n); for(i=0;i<n*8;i++) b[i/8]|=bits[i]<<(7-i%8);
}
static void des_ksched(const unsigned char key[8],unsigned char sk[16][48]){
   unsigned char kb[64],cd[56],c[28],d[28],t[28];
   int r,i,s;
   to_bits(key,8,kb); permute(PC1,56,kb,cd);
   memcpy(c,cd,28); memcpy(d,cd+28,28);
   for(r=0;r<16;r++){
      for(s=0;s<SHIFTS[r];s++){
         memcpy(t,c,28); for(i=0;i<28;i++) c[i]=t[(i+1)%28];
         memcpy(t,d,28); for(i=0;i<28;i++) d[i]=t[(i+1)%28];
      }
      memcpy(cd,c,28); memcpy(cd+28,d,28); permute(PC2,48,cd,sk[r]);
   }
}
```

```
static void des_crypt(const unsigned char in[8],unsigned char out[8],unsigned char sk[16]
[48],int dec){
    unsigned char b[64],ip[64],L[32],R[32],e[48],f[32],s[32],pre[64],ob[64];
    int r,i,j,row,col,v; unsigned char tmp;
    to_bits(in,8,b); permute(IP,64,b,ip);
    memcpy(L,ip,32); memcpy(R,ip+32,32);
    for(r=0;r<16;r++){
        const unsigned char *k=sk[dec?15-r:r];
        permute(EX,48,R,e); for(i=0;i<48;i++) e[i]^=k[i];
        for(j=0;j<8;j++){
            row=(e[6*j]<<1)|e[6*j+5];
            col=(e[6*j+1]<<3)|(e[6*j+2]<<2)|(e[6*j+3]<<1)|e[6*j+4];
            v=SB[j][row*16+col];
            for(i=0;i<4;i++) s[4*j+i]=(v>>(3-i))&1;
        }
        permute(PP,32,s,f);
        for(i=0;i<32;i++){ tmp=R[i]; R[i]=L[i]^f[i]; L[i]=tmp; }
    }
    memcpy(pre,R,32); memcpy(pre+32,L,32);
    permute(FP,64,pre,ob); from_bits(ob,8,out);
}
/* convenience: one 8-byte block with an 8-byte key */
static void des_block(const unsigned char in[8],unsigned char out[8],const unsigned char
key[8],int dec){
    unsigned char sk[16][48]; des_ksched(key,sk); des_crypt(in,out,sk,dec);
}
static void phex(const char *label,const unsigned char *b,int n){
    int i; printf("%s",label); for(i=0;i<n;i++) printf("%02X",b[i]); printf("\n");
}


#define NB 4   /* number of 64-bit blocks */
```

```
static void xor8(unsigned char *a, const unsigned char *b) { int i; for (i = 0; i < 8; i++) a[i] ^= b[i]; }


static void ecb(const unsigned char *in, unsigned char *out, const unsigned char *key, int dec) {
    int i; for (i = 0; i < NB; i++) des_block(in + 8 * i, out + 8 * i, key, dec);
}
static void cbc_enc(const unsigned char *in, unsigned char *out, const unsigned char *key, const unsigned char *iv) {
    unsigned char prev[8], x[8]; int i;
    memcpy(prev, iv, 8);
    for (i = 0; i < NB; i++) { memcpy(x, in + 8 * i, 8); xor8(x, prev); des_block(x, out + 8 * i, key, 0); memcpy(prev, out + 8 * i, 8); }
}
static void cbc_dec(const unsigned char *in, unsigned char *out, const unsigned char *key, const unsigned char *iv) {
    unsigned char prev[8], x[8]; int i;
    memcpy(prev, iv, 8);
    for (i = 0; i < NB; i++) { des_block(in + 8 * i, x, key, 1); xor8(x, prev); memcpy(out + 8 * i, x, 8); memcpy(prev, in + 8 * i, 8); }
}
static void report(const char *name, const unsigned char *good, const unsigned char *bad) {
    int i;
    printf("%s: ", name);
    for (i = 0; i < NB; i++)
        printf("P%d %s  ", i + 1, memcmp(good + 8 * i, bad + 8 * i, 8) ? "CORRUPTED" : "ok");
    printf("\n");
}


int main() {
    unsigned char key[8] = {0x13,0x34,0x57,0x79,0x9B,0xBC,0xDF,0xF1};
```

```
    unsigned char iv[8]  = {0,1,2,3,4,5,6,7};
    unsigned char P[33] = "BLOCK-01BLOCK-02BLOCK-03BLOCK-04";
    unsigned char C[32], D[32], P2[32], C2[32], D2[32];
    int i, diff;

    /* (a) error in transmitted C1 */
    printf("Part (a): bit error in the transmitted ciphertext block C1\n");
    ecb(P, C, key, 0);  C[0] ^= 0x01;  ecb(C, D, key, 1);
    report("  ECB", P, D);
    cbc_enc(P, C, key, iv);  C[0] ^= 0x01;  cbc_dec(C, D, key, iv);
    report("  CBC", P, D);
    printf("  -> In CBC, P1 is garbled and P2 gets a 1-bit error; P3, P4 are unaffected.\n");
    printf("  -> Blocks beyond P2 are NOT affected.\n\n");

    /* (b) error in the source plaintext P1 */
    printf("Part (b): bit error in the source plaintext P1 (before encryption)\n");
    memcpy(P2, P, 32); P2[0] ^= 0x01;
    cbc_enc(P, C, key, iv);
    cbc_enc(P2, C2, key, iv);
    for (diff = 0, i = 0; i < NB; i++) if (memcmp(C + 8 * i, C2 + 8 * i, 8)) diff++;
    printf("  Ciphertext blocks that change: %d of %d (error propagates to ALL later blocks)\
n", diff, NB);
    cbc_dec(C2, D2, key, iv);
    report("  Receiver (CBC)", P, D2);
    printf("  -> Receiver decrypts the stream correctly; only P1 (the original\n");
    printf("     erroneous block) is wrong, all other blocks are recovered fine.\n");
    return 0;
}
```

OUTPUT:

```
Part (a): bit error in the transmitted ciphertext block C1
  ECB: P1 CORRUPTED  P2 ok  P3 ok  P4 ok
  CBC: P1 CORRUPTED  P2 CORRUPTED  P3 ok  P4 ok
  -> In CBC, P1 is garbled and P2 gets a 1-bit error; P3, P4 are unaffected.
  -> Blocks beyond P2 are NOT affected.

Part (b): bit error in the source plaintext P1 (before encryption)
  Ciphertext blocks that change: 4 of 4 (error propagates to ALL later blocks)
  Receiver (CBC): P1 CORRUPTED  P2 ok  P3 ok  P4 ok
  -> Receiver decrypts the stream correctly; only P1 (the original
     erroneous block) is wrong, all other blocks are recovered fine.
```